

Open Web Application Security Project 2019 → 2023
non-profit, open source, top 10 in every field
API for interfacing backend to frontend

Cyber Attack :

Reconnaissance → Weaponize → Infiltration → Lateral Movement
→ Privilege Escalation → Breach

Api Attack : Find Vulnerability

Broken Object Level Auth

Broken Auth

Broken Object Property Level Auth

Unrestricted Resource Consumption

Broken Function Level Auth

Risk Ratings : Risk = Likelihood * Impact

Attack Vector :

Exploit API endpoints vulnerable to BOLA by manipulating obj-id

Security Weakness Description

Impacts Description

BOLA accessing unrestricted user info via api call w/o
Access Resources, Type | Valid | BOLA Test

OWASP Preventive Measures

- Auth with user policy and hierarchy
- Verify access to resource
- Random obj-id
- Test API for vulnerabilities

Broken - Authentication

- username, pswd, email, phone, etc
- ⊕ Check Attacker Vector Desc, Security Weakness Desc, Impact Desc

Weak Passwords, GraphQL Auth issues

Credential Stuffing

Predictable Tokens

Misconfig JSON Web Tokens (JWT)

Need Entropy in Auth

- ⊗ Rest needs API to be stateless

OWASP Preventive Measures

Broken - Object Property Level Authorization

Excessive Data Exposure

Mass Assignment

Returns more data than it should

More params than appl. intended

Object Properties, Response Validation

Unrestricted Resource Consumption

DDoS Attacks

Handle Technical & Financial Cost

Docker Controls, Limits, Max Payload

Boomer - Function Level Authorization

Edit or Modify other's data

Use else's privileges, unauth access

Unrestricted Access to Sensitive Data

Captha = Public Turing Test

Server Side Request Forgery

Access URI by API endpoints

InBand SSRF, Blind SSRF

Disable HTTP redirections, URL parser, sanitization

Security Misconfig

X-Powered-By Header

X-XSS-Protection Header

X-Response-Time Header

TLS, MITM, Default Acc., Testing Sec MisConfig

Improper Inventory Management

exposed unsupported APIs

Discovers Improper Inventory

URI and Header check; Document API details

Unsafe Consumption of API

Verify external or 3rd-party APIs

Encrypt channels and redirections

Injection

SQL, XML or any parsers

Sanitize inputs

Keep logs, Separate input data and commands, SIEM

Insufficient Logging & Monitoring

Identify abuse and anomaly

Business Logic Flaws

Exploit normal app functions

App subjective; Encoded payloads

Exposion API leak

Credentials

Bypass Web UI

Threat Modeling Approach, Bug-Bounty Program

1. Prompt Injection

Direct: Prompt input from user

Forget instructions, Exploit info

Rem Code, Priv Esc, Unauth Acc

Indirect: User $\leftrightarrow$ LLM $\leftrightarrow$ Web

Tailbreak from Web

Prevention: Priv Ctrl; Human in Loop; Segregate Content / Prompts
Trust Boundaries

2. Insecure Output Handling

App $\leftrightarrow$ LLM $\leftrightarrow$ DB

LLM might be compromised, check output

Prevention: LLM $\neq$ Trusted User; Validate IO; Guard Prompt

3. Training Data Poisoning

Synthesize from various sources; GIGO

Prevention: Know your sources, Verify info, Validate Results,
Wash-Rinse-Repeat

4. Over Reliance

Prone to Hallucinations

Prevention: Understand Limits; Train Users; Verify Sources



OWASP GenAI Security Project

Threats

- Samsung lost 'trade secrets' to ChatGPT
- Researchers expose weakness in co-pilot

Security Initiatives

Top 10 LLM and GenAI

AI threat intelligence

AI security governance

Secure AI adoption

Agentic App Security

Data Security

Red Teaming & Evaluation

AI Security Solution Landscape

Top 10: Prompt Injection

Data and Model Poisoning

Sensitive Info Disclosure

Supply Chain

Improper Output Handling

Excessive Agency

System Prompt Leakage

Vector and Embedding Weaknesses

Misinformation

Unbounded Consumption

Check Event Driven LLM Apps & Agentic AI structure

Security: Parameterization, Sanitation <<sys>> ... <</sys>>, [/INST]
Redbuff.ai, Heuristic Filtration, LLM-based Detection,
Vector DB, Dedicated Library.

MCP = standardization protocol for LLMs

Data Filters and Params on DB ; Output Sanitization

Pattern Matching, Regex, Filter, [REDACTED]

Implement Access Controls, Anonymization, Data Minimization

Rulesfile.mdc for payload attach

Filter is Allowed Command () && has Execution Permission ()
execute Sandboxed Command ()

RCE in Vanna-AI and RCE in LangChain

Cyber Security

Principles:

1. Defence in Depth
Security Mechanism - MFAuth, MDM or EDR, Firewall,
Testings, Encrypt
No SPOF, Failsafe
2. Principle of Least Privilege
Only, Harden, Priv concep, Just in Case vs Recapt
3. Separation of Duties
No SPOC, No Collision
4. Secure by Design
⑥ Prod, ① Reg Mts, ⑤ Test, ② Design, ④ Install, ③ Code
Start → Finish → After
Secure OOTB
5. Keep it Simple, Stupid
Complexity ≠ Security
Easier for good guy
6. Security by Obscurity
Secrecy ≠ Security
Neschoff Principle; open & observable
Glass box security, only key secret

Architecture

CTA:

Confidentiality

- Access Control (Authn & Authz)
- Encryption

Integrity

- Syslog (manip tamper)
- Digital Sign, Message Auth Codes
- Blockchain

Availability

- DDoS, Syn Flood, Reflection Attack

Roles and Tools :

Role & Mindset

- Architect, Engineers
- CyberS Arch thinks about how sys will fail

Tools

- Business Context Diagram
- System Context Diagram
- Arch Overview Diagram

Frameworks

- NIST CSF
- Lifecycle

Domains

- Monitor → Response

IAM: Identity Access Parameters

4A's: Administration, Authn, Authz, Audit

Store User Data in Directory

DB, Schema, Protocol (LDAP)

Multi-dir Sync (meta, index, virtual)

Provision and Revoking Priv System

Authn: Who? Know, Have, Are

Single Sign On System

Authz: What? RBA/AA - Loc, Reg Type, Reg Amt, Freq.

Priv Acc Mgmt (PAM)

Root, SysAdmin, DBA, NWA

Auto change pswd, keep logs and records

Audit

UBA / UEBA, Check logs, FED → Integrate with Others
Workforce, CTAM

Endpoint Security:

End Devices (Hw, Holistic, Business, Personal, Sw)

Controls, Policies & Patches ↓, Info & Alerts ↑

↳ Inventory (hw, sw), Sec Policy, Patching, Encryption,
Remote Wipe, Loc Tracking, AV/EDR, Disposal

BYOD/IT/c

↳ Consent, Software, Hardware, Services

Make it easier to do the good thing

Network:

Firewall - Packet Filtering, Stateful Pkt Insp, Proxy,
Net Addr Trans, TLS/SSL

NAT ↔ DHCP : Int & Ext Traffic

Segmentation - Bastion, TorHomed, Basic DMZ, MultiTier DMZ

VPN - Secure Channel → Untrusted Net

Encryption → Confidentiality

Limited INSP

SSH, TLS/SSL, IDSEC, PPTP/L2TP

Broad → App Specific for Granularity and Control

SASE - Secure Access Service Edge

= (NetSec + WAN) / cloud + Authn & Authz

FW+Sec WG+DLP Software Defined Scale+Elastic+Agile

Appl. Security:

Sw → Bugs → Vulnerability

More injection in Coding Phase; Finds in Testing Phase ⇒ Costlier

Traditional SDLC is Linear

DevOps = Cyclical, Integrated, Rapid, Agile

DevSecOps - security at each phase, by design

Secure Coding

- Checklist, Coding Practices, Trusted Libs, Standard Aochs,
- Avoid Mistakes, Software Bill of Materials (Supply Chain)

Log4J vulnerability

Vulnerability Testing

- SAST : White Box, Source Code, Earlier detect
- DAST : Black Box, Running App, Later detect

} Both/
AND

Data Security :

Governance - Policy, Classify, Catalog, Resilience

Discover - DB Structure, File Unstor, Network, DLP

Protect - Encrypt (Rest/Motion), Key Mgmt, Acc Cntrl, Backup

Quantum Safe Crypto

Compliance - Report, Retain

Detection - Monitor, UBA, Alerts

Response - Cases, Dyn PBs, Orch, Auto

Top 5 : AI, DevSecOps, Incident Response, Cryptography,
Emp/User Training

Detection:

Security = Prev + Det + Resp

SIEM - Correlate, Analyze, Rules / Anomalies / Trends

XDR - TopDown EDR, Federated Search

SIEM + XDR

Hunt: Reconnaissance → Attack → MTTI → Aware → MTTC
(check site) (mean time to identify) (mean time to contain)

Threat Hunting = Find Attack earlier, use XDR tools, Pro Act

Exp + Instinct = Hypothesis + Tools = Early Detect

MTTI ≈ 200 days, MTTC ≈ 70 days

Response:

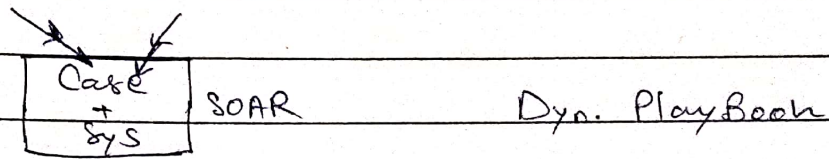
SOC → Center : Monitor, Detect, Respond

IR → Traditional : Manual, Triage, Remediate

SOAR → Modern : Orch, Auto

Cases :

Attach → SIEM → XDR // Added Artifacts for headstart



Create, Modify, Assign, Triage, Investigate, Remediate

FOAM → Manual SOAR

Later Orch & Auto

Breach Notification :

Data, Geography, Regions Regulatory Reg, ...